

DNA Computing

From Molecules to Algorithms

Undergraduate edition

Chapter 1

What does it mean to compute?

A coordinated textbook series from Hermon AI

Internally reviewed development chapter · 6 September 2026

How to use this preview

This is the first chapter of a new undergraduate edition, not a completed textbook. Later chapter references point to the approved teaching plan. Earlier editions remain separately preserved.

Read pictures with their captions, try worked examples by hand, and attempt exercises before opening the answers. The Python section is optional. Linked glossary entries return to the section teaching each term; the index points to substantive definitions.

Figures are original editable vector artwork with Unicode TXT companions. Their captions state the simplifications. This preview contains no new laboratory result or claim of a research-model advantage.

Review so far is internal author-agent review using beginner and specialist perspectives. It is not independent expert certification, a learner study or an accessibility-conformance certification. The repository includes sources, tests, build instructions and a candid production report.

Contents

1	What does it mean to compute?	1
1.1	Start with five counters, not five prerequisites	1
1.2	A rule must say what to do	2
1.3	Optional: let a computer follow that rule	3
1.4	Information needs a representation	4
1.5	Why introduce DNA?	5
1.6	From a physical change to an answer	6
1.7	What the rest of the book must earn	7
1.8	Practice: recognize, apply, question	8
1.9	Answers and a second look	9
	Chapter 1 glossary	10
	Bibliography	12
	Index	13

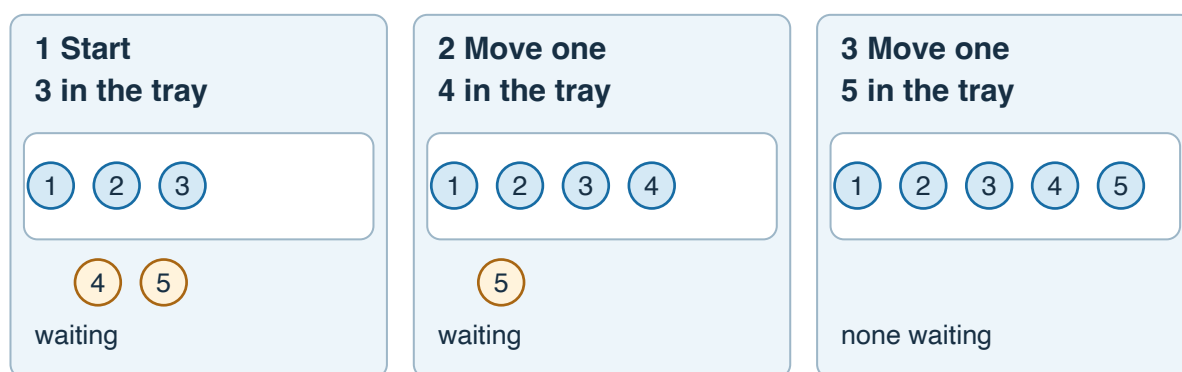
CHAPTER 1

What does it mean to compute?

1.1 Start with five counters, not five prerequisites

A calculator gives answers. So does a person moving counters on a table. Could a molecule take part in the same kind of activity? Before we ask what DNA can do, we need to decide what would count as computing.

Put three counters in a tray and leave two beside it. Move one waiting counter into the tray. Then move the other. How many are in the tray now? Figure 1.1 keeps the counters numbered so that you can follow the objects, not just trust the final number.



Same five objects throughout; only their location changes.

Figure 1.1: Adding two by moving two objects. The numbers identify counters; they do not change during the operation. Amber counters are waiting outside the counted group. No objects are created or destroyed.

Input is the starting information supplied to a computation. Here it is the count three. The **output** is the resulting information we read from it. Here it is the count five. A **step** is one specified action in a procedure. Moving one waiting counter is a step.

KEY IDEA

For this opening, **computation** means carrying out specified rules on represented information to obtain a result. We will examine each part of that description. A result need not be a number: it could be a word or a yes-or-no answer.

By the end of this chapter, you should identify a starting value, a rule and a result; explain why a representation needs a reading convention; and say what is needed to interpret a molecular process as an answer. You need only ordinary counting and careful reading.

1.2 A rule must say what to do

Suppose two people receive the same three counters and the instruction “add more.” One adds one counter. The other adds two. Neither has disobeyed the words, yet their answers differ. The problem is not the material. The instruction left something undecided.

Rule is an explicit instruction specifying an allowed operation. For our example, write: “Start with the supplied count; add exactly two; then stop.” The stopping instruction matters. Applying the rule twice would add four, which is a different task.

UNCLEAR: start at 3; add more

Add one: 4 or add two: 5 — both follow the words.

make the rule precise



EXPLICIT: start at 3; add exactly 2

One completed application gives 5, not 4 and not 7.

This example specifies one rule and one application.

Figure 1.2: Make the intended operation explicit. The upper instruction leaves the amount open. The lower one specifies an amount and one completed application. This example has one intended result; later definitions will cover a wider range of computational processes.

Worked example: start at three. First, record the input: three. Second, add one to obtain four. Third, add one again to obtain five. Finally, stop and read the count. The two movements together implement the instruction “add exactly two.”

We can now shorten the story using familiar arithmetic:

$$3 + 2 = 5.$$

The left side says what to do with the starting count; the right side records the result. The plus sign asks for addition and the equals sign says that both sides name the same number. The equation does not tell us whether we used counters, pencil marks or a calculator.

TRY IT

Begin with four counters and apply the same rule once. A first move gives five; what does the second move give? Say which part changed: the input, the rule, or both.

The result is six. Only the input changed. A useful rule can be reused without becoming a different rule each time.

1.3 Optional: let a computer follow that rule

You can understand this chapter without running software. This short example simply makes the previous instruction executable. A **program** is instructions written in a form a computer can carry out. **Python** is the programming language used for these examples. A programming language supplies agreed ways to write instructions.

Before looking at the code, read its procedure: receive a count; add two; give back the result. Nothing is learned from examples here, and nothing random is chosen.

```
def add_two(count):
    return count + 2
```

A **function** is a named, reusable rule in this program. The word `def` begins its definition. Its name is `add_two`. The name `count` stands for the supplied value; it is not a fixed value of three. Parentheses enclose that input name. The colon begins the indented body. The line beginning `return` gives back the result of adding two.

Asking for `add_two(3)` gives 5. Asking for `add_two(4)` gives 6. This notation names the rule and places the chosen input inside parentheses. The computer need not move counters; it carries out the written addition.

Starting count	Add exactly two	Output
0	$0 + 2$	2
3	$3 + 2$	5
4	$4 + 2$	6

This table is generated by running the source file's function on the three stated inputs. The first row starts at zero, so it also checks an empty tray. It is a record of these cases, not a proof about every possible input.

To try it, open Python 3, type the two printed lines, press Enter on a blank line to finish the definition, and then type `add_two(3)`. The official tutorial explains definitions and return in greater depth [1].

COMMON MISTAKE

"The program ran" and "the program follows the intended rule" are different claims. Check the result against a hand calculation. Repeating the same mistake reliably would not make it correct.

We use whole-number counts of zero or more. The tiny function does not check the input type: a word or another unsuitable value is outside this exercise. Rejecting invalid inputs belongs in the programming chapters.

1.4 Information needs a representation

Three counters, the written numeral 3 and three marked squares look different. Yet each can answer the same question: how many? The agreement depends on how we read each display.

Information here means what a chosen description tells us about the possibilities relevant to a task. A count of three distinguishes that quantity from two or four. This is an intuitive starting point, not a formula for measuring information.

Representation is a chosen way to express information through distinguishable forms. A **physical medium** is the material carrier of a representation. Ink on paper and counters on a tray are different carriers.

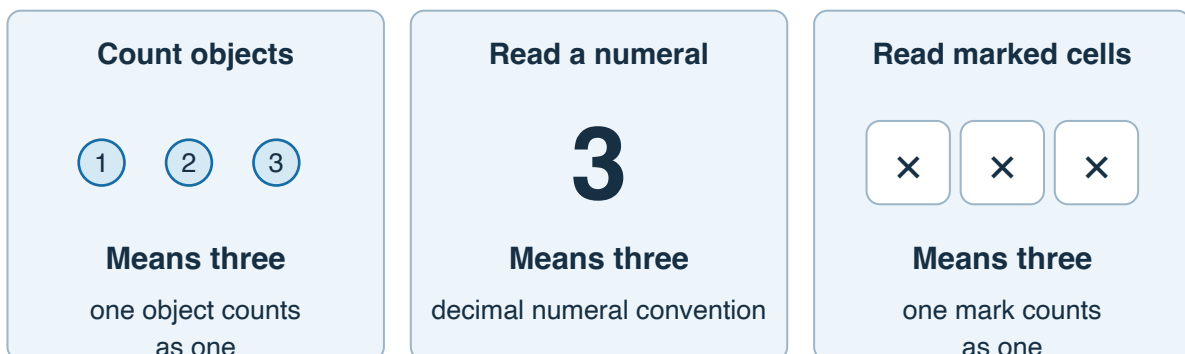


Figure 1.3: Three representations of the same quantity. Read the convention under each panel: count objects, read a numeral, or count marked cells. “Decimal” means the familiar number-writing system with ten digits, 0 through 9. Shape alone does not establish meaning.

In the first panel, each counter contributes one to the count. In the third, each mark contributes one. The numeral uses a different convention: one printed character can stand for the whole quantity. A reader who does not know that convention cannot recover the intended answer merely by inspecting the ink.

Worked comparison. To apply “add two” to counters, place two more in the counted collection. To apply it to a written numeral, read the number, perform the addition and write the resulting numeral. The physical actions differ, while the intended numerical transformation agrees.

COMMON MISTAKE

A mark has no universal meaning by itself. Three marks could mean three items, three completed steps, or an agreed code. State the reading convention before calling a physical pattern an answer.

This gives us a sharper question about DNA: could its distinguishable forms represent a problem, and could controlled changes support a rule for solving it?

1.5 Why introduce DNA?

Matter has structure below the scale we can see. An **atom** is a small constituent of matter, such as a carbon atom. A **bond** is a chemical connection that holds atoms together. A **molecule** is a group of atoms joined by chemical bonds. These are working descriptions; Chapter 7 will explain the chemistry.

DNA is deoxyribonucleic acid, a molecule that carries genetic information in living organisms. Its structure includes ordered building blocks with four kinds of parts conventionally labeled A, C, G and T [2]. A **strand** is one connected chain in DNA. A **sequence** is an ordered list; a DNA sequence records the order of those letter-labeled parts.

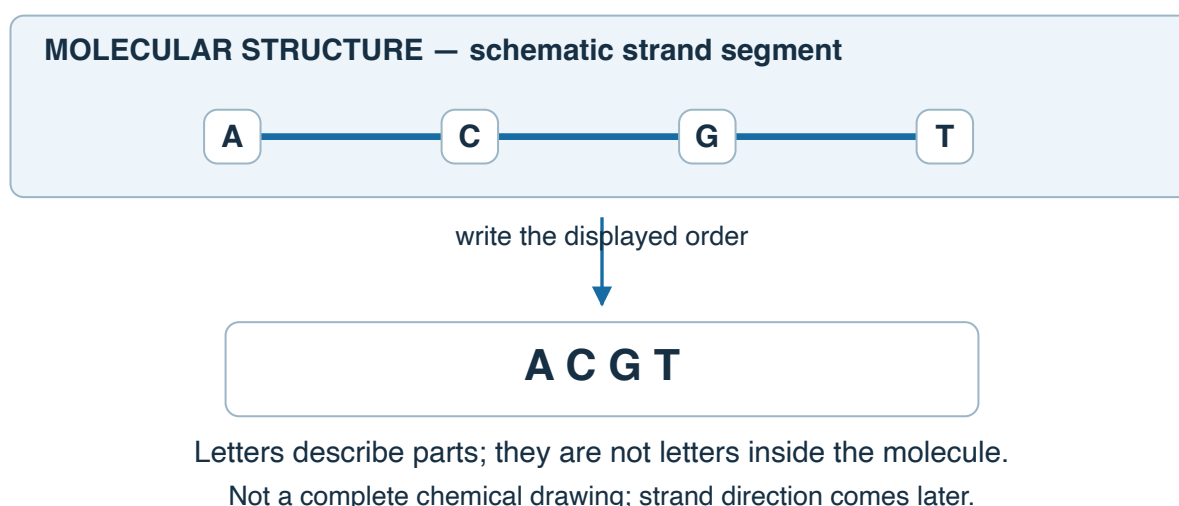


Figure 1.4: A short strand segment and a written description of its order. The line means connected structure, not movement. Each box stands for a building block distinguished by its letter-labeled part; it is not an atom. Original schematic supported by [2], not a chemical structure or a drawing to scale.

The drawing leaves things out. It does not show individual atoms, the full arrangement of DNA's strands, or biologists' direction conventions. Chapters 10 and 11 supply those details. Retain one observation: an ordered physical structure can have a written description.

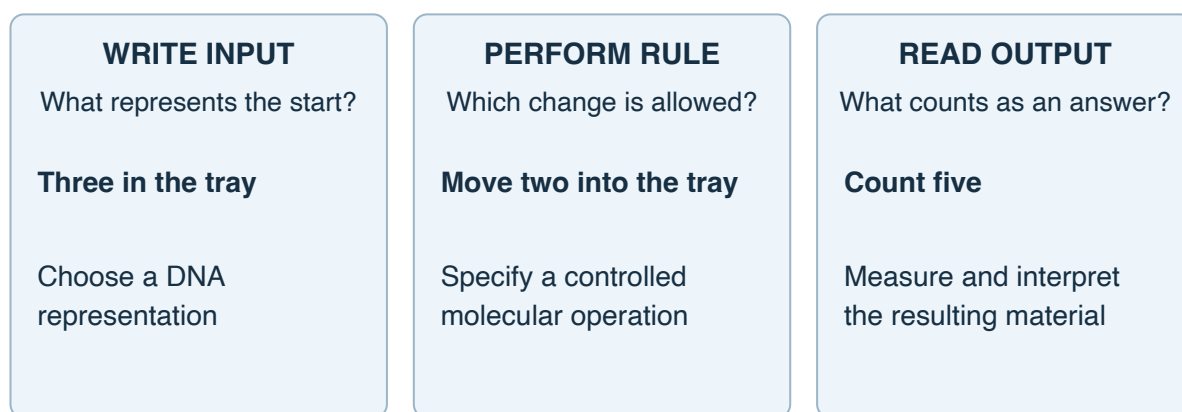
Change the displayed order from ACGT to AGCT and the description changes. That does not assign either sequence a numerical meaning. For an invented task, a designer must say what each sequence represents. Biological information and a designer's encoding of arithmetic are not automatically the same thing.

DNA combines ordered structure with physical chemistry. But a representation alone does not solve a problem. We must also specify what happens to the material and how to read the result.

1.6 From a physical change to an answer

A counter falls from a table. Something physical happened, but that alone does not mean our addition problem was solved. If the rule was “add two to the tray,” the fall is probably an error. We need a connection between the task, the material and the reading.

DNA computing means using designed DNA-based physical processes to carry out computational tasks. Figure 1.5 gives three questions to ask of such a design. The lower lane contains questions, not a recipe. No chemical operation is established by drawing a box.



Top example: counters. Bottom lane: questions for a DNA design.

Figure 1.5: A design checklist, not a laboratory result. The counters supply a complete small case. A DNA design must separately establish its representation, operation and interpretation. Left-to-right order organizes the explanation; it does not assert a particular reaction mechanism.

Try the checklist on counters. First, three in the tray represents the input. Second, moving exactly two waiting counters into it performs the rule. Third, counting all counters inside gives the output. If the reader counts only the original three, even perfect movements produce a wrongly read answer.

A molecular design faces corresponding obligations but different practical difficulties. How is the starting material prepared? Which changes are permitted? What observation distinguishes a wanted result from an unwanted one? We will learn the tools needed to answer these questions before inspecting detailed experiments.

There is historical evidence beyond metaphor. In 1994, Leonard Adleman reported a small laboratory demonstration addressing a route-finding problem using molecular biology [3]. This does not establish that arbitrary large problems become easy. Later chapters teach the problem before inspecting the experiment.

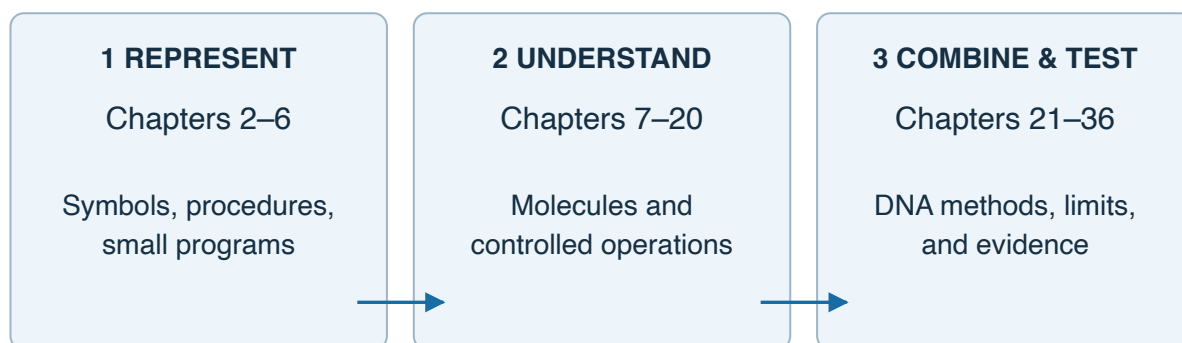
RESEARCH FRONTIER

Separate a proposed rule, a simplified model and a laboratory demonstration. This chapter's counters and Python are teaching examples. Its DNA workflow is a design abstraction, not a new experiment or a claim of computational advantage.

1.7 What the rest of the book must earn

We have not explained how to build a DNA computer. We have made the question meaningful: what information does a material represent, what rule does an operation implement, and how does an observation become an output?

The next chapter, *Symbols, information and representations*, develops reading conventions. A symbol is a distinguishable mark used by convention; Chapter 2 asks how symbols can be combined without confusing the reader. We then build the computing and molecular foundations before bringing them together.



Arrows: teaching order. These later chapters are still planned.

Figure 1.6: The planned teaching route. Arrows mean “learn this before relying on it,” not a biological process. First develop descriptions and procedures, then molecular understanding, then combine them and examine evidence. Only Chapter 1 is produced in this undergraduate preview.

Established: the addition follows a stated rule; its printed code can be checked against that rule. DNA has ordered structure, supported by the cited biology source. The historical experiment supports a bounded claim that molecules have been used in computation.

Simplified: our definition of information is informal and the DNA picture hides chemistry. Neither is a complete theory.

Analogy: counters help us ask about representations and operations. They do not tell us which chemical operation will work.

Research question: which DNA-based methods are useful for which tasks, under which constraints? Answering that requires more than an appealing picture or one correct small output.

CHECK YOUR UNDERSTANDING

Explain the chapter without DNA first: “Choose a representation, carry out an explicit rule, read the result.” Then explain what DNA changes: its physical representation and available operations require scientific justification.

1.8 Practice: recognize, apply, question

Try the questions before reading the answers. None requires a laboratory or a university science course.

1. **Recognize.** A card says “start at four; add two once; stop.” Name the input, the rule and the output. Which number is not the input?
2. **Repair.** Why might “make the count bigger” yield different results? Rewrite it so that everyone starting at four should obtain six.
3. **Read.** Four marked cells appear on a page. Under a one-mark-per-item convention, how many items are represented? If the convention is unstated, must four be the meaning?
4. **Trace.** Start with one counter in the tray and two waiting outside. Describe each move. How many objects exist before and after? Distinguish the tray’s count from the total.
5. **Optional code.** Predict `add_two(7)`. Someone changes the function to add three but leaves its name unchanged. Why is its name insufficient evidence of correctness?
6. **Separate description from meaning.** Two written DNA sequences are ACGT and AGCT. What changed? What agreement is needed to say that one represents three?
7. **Inspect a claim.** A diagram shows “DNA input; molecular operation; answer.” Its author says this proves efficient solutions to a large problem. Name two things the diagram has not established.
8. **Exit explanation.** In three or four sentences, explain how computation can use physical material without making every physical event a successful computation.

BEFORE YOU CHECK

Point to the sentence or picture that helped. If you can only repeat a term, replace it with an example. Understanding representation includes stating how a particular mark is read.

A successful exit explanation identifies a task, a representation, an operation following a rule and a way to read its result. It also identifies a possible failure. Vocabulary without those connections is not yet the goal.

1.9 Answers and a second look

1. Input: four. Rule: add exactly two once, then stop. Output: six. Two specifies the amount added, not the starting count.
2. "Bigger" leaves the amount open. "Add exactly two once, then stop" removes that ambiguity. Also specify which collection to count.
3. Four under the stated convention. Without it, four is possible, not guaranteed. The marks could describe something else.
4. Initially one inside, two outside. After one move: two inside, one outside. After the second: three inside, none outside. The total remains three. Only location and the tray's count change.
5. Nine. A name is a label, not a guarantee. After the edit, hand cases would disagree with the intended add-two rule. Inspect the body and check known cases.
6. The second and third positions exchange C and G. The order changed. A mapping from sequence to number is still needed; the letters alone do not imply three.
7. The diagram establishes neither that its operation is physically achievable nor that readout is correct. It also gives no measurements of time, material or error as problem size grows. Any two explained gaps suffice.
8. One answer: "I represent a count by counters inside a tray. I move two more into it, then count the result. A counter falling away changes the material but violates the intended operation. A molecular computation likewise needs a stated connection between process and interpreted answer."

Return to the opening. Could a molecule compute? A designed molecular process can participate in computation when its representation, operation and readout support a specified task. DNA's structure makes it a candidate, but the proposed operation still needs evidence.

Keep the question you can now ask separate from the mechanism you have not learned. Chapter 2 develops reading conventions; it does not presume that one DNA picture has taught molecular biology.

Chapter 1 glossary

atom

is a small constituent of matter, such as a carbon atom. First taught in Section 1.5; see also [bond](#).

bond

is a chemical connection that holds atoms together. First taught in Section 1.5; see also [atom](#).

computation

means carrying out specified rules on represented information to obtain a result. First taught in Section 1.1; see also [rule](#).

DNA

is deoxyribonucleic acid, a molecule that carries genetic information in living organisms. First taught in Section 1.5; see also [molecule](#).

DNA computing

means using designed DNA-based physical processes to carry out computational tasks. First taught in Section 1.6; see also [DNA](#).

function

is a named, reusable rule in this program. First taught in Section 1.3; see also [program](#).

information

here means what a chosen description tells us about the possibilities relevant to a task. First taught in Section 1.4; see also [representation](#).

input

is the starting information supplied to a computation. First taught in Section 1.1; see also [output](#).

molecule

is a group of atoms joined by chemical bonds. First taught in Section 1.5; see also [DNA](#).

output

is the resulting information we read from it. First taught in Section 1.1; see also [input](#).

physical medium

is the material carrier of a representation. First taught in Section 1.4; see also [representation](#).

program

is instructions written in a form a computer can carry out. First taught in Section 1.3; see also [function](#).

Python

is the programming language used for these examples. First taught in Section 1.3; see also [program](#).

representation

is a chosen way to express information through distinguishable forms. First taught in Section 1.4; see also [information](#).

rule

is an explicit instruction specifying an allowed operation. First taught in Section 1.2; see also [computation](#).

sequence

is an ordered list; a DNA sequence records the order of those letter-labeled parts. First taught in Section 1.5; see also [strand](#).

step

is one specified action in a procedure. First taught in Section 1.1; see also [rule](#).

strand

is one connected chain in DNA. First taught in Section 1.5; see also [sequence](#).

Bibliography

- [1] Python Software Foundation. *The Python Tutorial: More Control Flow Tools*. Sections on if statements, functions and return.
<https://docs.python.org/3/tutorial/controlflow.html>. Accessed 6 September 2026.
Examples tested locally with Python 3.13.6.
- [2] National Human Genome Research Institute. *Deoxyribonucleic Acid (DNA)*.
<https://www.genome.gov/genetics-glossary/Deoxyribonucleic-Acid-DNA>. Accessed 6 September 2026.
- [3] Leonard M. Adleman (1994). Molecular computation of solutions to combinatorial problems. *Science* 266(5187), 1021–1024. doi:10.1126/science.7973651.
<https://pubmed.ncbi.nlm.nih.gov/7973651/>. This chapter uses the abstract’s bounded historical claim, not a reconstruction of the experiment.

Index

atom, 5

bond, 5

computation, 1

DNA, 5

DNA computing, 6

function, 3

information, 4

input, 1

molecule, 5

output, 1

physical medium, 4

program, 3

Python, 3

representation, 4

rule, 2

sequence, 5

step, 1

strand, 5